

Capítulo 8: Abstracciones Contextuales (Scala 3: `given` y `using`)

1. Explicación Teórica: La Inyección de Dependencias a Nivel de Tipo

Las Abstracciones Contextuales son una característica central de Scala 3 (Proyecto Dotty), diseñadas para **abstraer el contexto** de una operación, permitiendo que el compilador infiera y sintetice automáticamente los "términos" o valores que una pieza de código necesita.

Este mecanismo reemplaza y moderniza la palabra clave `implicit` de Scala 2, que era a menudo utilizada para múltiples propósitos (parámetros, conversiones, *Type Classes*), lo que generaba confusión y código difícil de rastrear.

La filosofía de Scala 3 se centra en la **intención** en lugar del mecanismo subyacente:

- `given` (**Provisión**): La intención es **definir** un valor canónico o una instancia canónica de un tipo particular que está disponible en el contexto. Este valor es a menudo una implementación de un patrón de *Type Class* o una dependencia compartida.
- `using` (**Consumo**): La intención es **declarar** que una función o método requiere un valor del contexto circundante, y el compilador debe inyectarlo automáticamente. Este concepto es la base de la inyección de dependencias funcional en Scala.

En esencia, las Abstracciones Contextuales le dicen al compilador: "Si puedes **probar la existencia** de un valor de este tipo, úsalo aquí".

Aplicaciones Clave

Este mecanismo unificado resuelve varios problemas de diseño de software de alto nivel:

1. **Implementación de Type Classes:** La forma canónica de añadir comportamiento a tipos que no posees (como añadir lógica de ordenación a `Int` o lógica de serialización a `case class`).
2. **Inyección de Dependencias:** Proporcionar dependencias o configuraciones de manera segura en tipos (ej. un `ExecutionContext` o una conexión a base de datos).

2. Sintaxis y Estructura Funcional: `given` y `using`

La sintaxis de Scala 3 para las abstracciones contextuales es limpia y explícita, a menudo utilizando la nueva sintaxis de indentación para la implementación de las instancias.

A. Definición de la Instancia (`given`)

Un `given` se utiliza para definir la implementación de un *Type Class* o el valor de una dependencia.

Sintaxis:

```
1  given [NombreOpcional]: Tipo[Parámetros] with // Para instancias con
    cuerpo
2    // Implementación de métodos abstractos
3    def metodoAbstracto(args): Retorno = ...
4
5  given [NombreOpcional]: Tipo[Parámetros] = Expresión // Para instancias
    simples o alias
```

Ejemplo realista (Type Class: Ordenación por Defecto):

Modelaremos una `Persona` y definiremos una ordenación estándar por su apellido, utilizando el *Type Class* `Ordering[T]` de la biblioteca estándar.

```
1  case class Persona(nombre: String, apellido: String, edad: Int)
2
3  // Definimos una instancia 'given' de Ordering para el tipo Persona.
4  // El nombre 'StandardPersonOrdering' es opcional pero útil.
5  given StandardPersonOrdering: Ordering[Persona] with
6    // Implementación del método compare (similar a Java's Comparator)
7    override def compare(x: Persona, y: Persona): Int =
8      x.apellido.compareTo(y.apellido) // Ordena alfabéticamente por
        apellido
```

B. Consumo del Contexto (`using`)

La cláusula `using` se añade a la firma de una función o método, indicando que se espera que el compilador resuelva e inyecte un valor de ese tipo de la forma más concisa posible.

Sintaxis (Context Parameter):

```
1  def nombreFuncion[T](param1: T, ...)(using dependencia: TipoDependencia):
    Retorno = ...
```

Ejemplo realista (Consumo de Type Class):

Definimos una función genérica `ordenarLista` que requiere un `Ordering[T]` en el contexto para poder ordenar cualquier tipo `T`.

```
1  // La función requiere un valor de tipo Ordering[T] que será proporcionado
    por un 'given'
```

```

2 // El parámetro se nombra 'orden' dentro del cuerpo del método.
3 def ordenarLista[T](lista: List[T])(using orden: Ordering[T]): List[T] =
4   // Aquí usamos el método 'sorted' de List, que internamente también
   utiliza 'using'
5   // o podemos usar 'orden' directamente:
6   lista.sortWith((a, b) => orden.compare(a, b) < 0)
7 // Si la sintaxis fuera más concisa (Context Bounds):
8 // def ordenarLista[T: Ordering](lista: List[T]): List[T] = lista.sorted
9
10 // Uso:
11 val listaPersonas = List(
12   Persona("Ana", "Smith", 30),
13   Persona("Bob", "Jones", 25)
14 )
15
16 // El compilador inyecta automáticamente el 'given StandardPersonOrdering'
   definido arriba.
17 val listaOrdenada = ordenarLista(listaPersonas)
18 // listaOrdenada: List(Persona(Bob, Jones, 25), Persona(Ana, Smith, 30))

```

C. Derivación de Instancias (given dependiendo de using)

Una de las capacidades más poderosas es la **derivación contextual**: un `given` puede depender de la existencia de otros `given` s para poder ser definido. Esto se logra mediante una cláusula `using` dentro de la definición del `given`, creando una "fábrica de instancias".

Ejemplo realista (Derivación de Option):

Si puedes ordenar el tipo `T` (tienes `Ordering[T]`), puedes crear automáticamente un `Ordering[Option[T]]` (lógica para ordenar listas que contienen valores ausentes).

```

1 // Deriva Ordering[Option[T]] si ya existe un Ordering[T] en el contexto
   (usando)
2 given optionOrdering[T](using ord: Ordering[T]): Ordering[Option[T]] with
3   override def compare(x: Option[T], y: Option[T]): Int =
4     (x, y) match
5       case (Some(a), Some(b)) => ord.compare(a, b) // Si ambos existen,
       usa el Ordering[T]
6       case (None, None)      => 0                  // Si ambos son None,
       son iguales
7       case (None, _)         => -1                  // None es menor que
       Some (convención)
8       case (Some(_), None)   => 1                  // Some es mayor que
       None

```

Si el compilador ve una llamada a `ordenarLista` con un `List[Option[Persona]]` , seguirá la cadena: necesita `Ordering[Option[Persona]]` -> encuentra `given optionOrdering` -> este necesita `Ordering[Persona]` -> encuentra `given StandardPersonOrdering` -> lo usa y resuelve la instancia para `List[Option[Persona]]` .

3. Comparativa con Java/Python

Aspecto	Java (o Spring DI)	Python (Dinámico)	Scala 3 (<code>given</code> / <code>using</code>)
Resolución/Inferencia	Manual o por <i>runtime</i> (Reflection, anotaciones <code>@Autowired</code>).	Manual (inyección de argumentos) o <i>runtime</i> (meta-clases).	Automática en tiempo de compilación (Term Inference). El compilador busca el tipo necesario.
Type Classes	Interfaces (ej. <code>java.util.Comparator</code>). La implementación se pasa manualmente.	No existe un concepto directo.	Mecanismo nativo (<code>trait</code> + <code>given</code>) que inyecta la implementación automáticamente.
Ambigüedad	Si dos beans del mismo tipo están en el contenedor de Spring, se requiere calificación manual.	No aplica (dinámico).	Prevenida en compilación: Solo puede haber una instancia <code>given</code> de un tipo específico en un alcance (scope) dado.
Conversiones	<i>Type Erasure</i> . Requiere <i>casting</i> en tiempo de ejecución.	Tipado dinámico.	El enfoque de Scala 3 hace que las conversiones (<code>Conversion</code>) sean explícitas y se importen de forma clara, reduciendo el riesgo de errores furtivos (<i>sneaky bugs</i>).

4. Best Practices (The Scala Way)

1. **Prioriza `given` y `using`** : En el código moderno de Scala 3, utiliza las Abstracciones Contextuales para la inyección automática en lugar del mecanismo `implicit` de Scala 2 (aunque `implicit` sigue siendo compatible en la mayoría de los casos). La nueva sintaxis de Scala 3 mejora la claridad del 95% de los programas que usan abstracciones contextuales.
2. **Un solo `given` por Tipo**: Asegúrate de que solo exista **una** instancia `given` de un tipo determinado (`Ordering[Int]` , `ExecutionContext` , etc.) en el ámbito donde se llama a la función, para evitar ambigüedades en la inyección. Si hay una ambigüedad, el código **no compilará**.
3. **Inferencia vs. Nombramiento**:
 - Si el `given` se va a usar solo para ser inyectado automáticamente (ej. una instancia de `Type Class`), puedes omitir el nombre y solo usar `given Tipo with...` .
 - Si necesitas referenciar el valor `given` explícitamente dentro de tu código (ej. `given ExecutionContext` para usarlo en un método que requiere `using`), debes nombrarlo (`given ec: ExecutionContext = ...`).
4. **Uso para Restricción de Tipos (*Type Bounds*)**: Utiliza la sintaxis concisa de límites contextuales (`[T: TypeClass]`) si la única intención es **restringir** qué tipos pueden llamar a tu método, sin que la función necesite acceder al valor inyectado internamente. Esto es azúcar sintáctico para una cláusula `using` que no nombra el parámetro.
5. **Importación Explícita**: Cuando se importan `given` instances desde otro paquete, se recomienda importarlos explícitamente (`import paquete.dadoConNombre`) o por tipo (`import paquete.given Tipo[T]`). Esto ayuda a los desarrolladores a saber exactamente *de dónde viene la magia*, resolviendo el problema de rastreo de las antiguas *implicit*s.